CS130: Project Idea, Design Doc, Presentation

Deadline: Friday, 04/24 11:59pm

Objective

In this part of the project, your team will propose, design, and justify a software application that you will build during the rest of the quarter. The purpose is to help your team choose a project that is meaningful, feasible, and well-designed before implementation begins.

Your project may be a web app, mobile app, desktop app, browser extension, game, tool, or another interactive software system. You may use any programming language, framework, or platform, as long as the choice is justifiable.

We encourage a broad range of project ideas. **A strong project does not need to be completely unprecedented but we strongly appreciate the idea's novelty.** It should instead solve a real problem for a clearly identified set of users, improve meaningfully on an existing workflow, or explore a thoughtful new combination of features, interaction design, or technical approach. Our goal is to help you gain insights into the process of software design and development by working on this project.

First, you will start by identifying the requirements from a user perspective. Then you can design a solution in the form of an application to answer the user requirements and create a working prototype of the proposed application. Finally, you may identify key issues around and improve the prototype into a useful product.

Project Idea

A strong project idea usually has the following properties:

- It addresses a concrete **user need** or pain point rather than being only technology-driven.
- It has a clearly scoped **core feature set** that can be implemented in about six weeks by your team.
- It has enough design **depth/complexity** to support discussion of requirements, architecture, data model, and user interaction.
- It is **testable** and **demo-able**. Your team will show realistic usage scenarios of your application in the final presentation and demo video.

Projects are not expected to be startup-scale products. A smaller, polished, coherent system with thoughtful design is usually better than a large but shallow system with too many disconnected features.

Collaboration

All team members are expected to contribute meaningfully to the project throughout the quarter. We expect responsibilities to be visible, coordinated, and **reasonably balanced over time**.

Healthy collaboration includes planning work in advance, communicating blockers early, reviewing one another's work, and making design decisions collectively when appropriate. Teams should **avoid siloing the entire project into isolated individual pieces** with little integration.

To support effective collaboration, all substantive **code changes must be made through pull requests** rather than being pushed directly to the main branch. In addition, pull requests **must receive code review** from at least one other team member before being merged. Code review should be used to discuss design decisions, catch bugs, improve code quality, and ensure that knowledge about the codebase is shared across the team.

Design Doc

The design doc is your team's proposal for what you plan to build and why it is a good project for this course. It should clearly communicate the problem you are solving, the users you are designing for, the requirements and design of your system, and why the project is feasible within the course timeline. A strong design doc is not just a description of features. It should demonstrate that your team has thought carefully about scope, design decisions, technical risks, and how the proposed system will be implemented and evaluated.

Design Doc Requirements & Structure

Your design doc should include the following sections:

Title Page
Project Name:
Team Name:
Team Members (names and UIDs):
GitHub Repository URL:

Context / Scope
This section shall include a description on why the proposed application is needed by users, what the problem is, and what your vision is for creating the application. It shall also include a list of background information about the domain that is relevant to understand the remaining sections.

Goals & Non-Goals

1. This section **shall** include list of **user stories** (each with list of sufficient acceptance criteria in the to clearly explain the functional requirements). Please pay attention to the INVEST criteria when writing your user stories. If user stories depend on each other, please make these dependencies **explicit**. You need **at least 4 user stories** for your design doc.
2. This section **shall** include a **UML use case diagram** to visualize the user stories & their actors, and the relationships between the user stories. You *may* also include UML state machine diagrams to specify behavioral requirements if you believe it is helpful.
3. You **shall** specify other non-functional requirements such as performance, reliability, as (simplified) Quality Attribute Scenarios
4. Please include mock-up screenshots of the proposed application and describe the associated user interaction scenarios in detail, so that the teaching team can understand the proposed features and assess the difficulty level of the proposed implementation as best as possible (e.g., similar to how user manuals are written.)

The Design
This section should describe the design you plan to implement. You should include:

- A high-level description of your design
- At least one **sequence diagram** of interesting integrations of components within your system
- At least one class diagram **or** state machine diagram of your high-level system design
- Descriptions of the relevant **APIs** of your system. Please describe the semantics of the interface as well (i.e., what does the operation do? What are relevant pre- and post-conditions of the operations? What errors might occur? What meaning do the parameters have and what units are expected?)
- A description of the **data model** of your system
- A description of the tech stack you plan to use (e.g., programming language, frameworks, libraries, technology for data management

Alternatives Considered
This section should describe viable other options you did *not* select and describe good arguments why you did not select this option. Please do not list “strawman” alternatives (i.e., options that are obviously worse than the design you described), but *good* alternatives that are viable candidates (and ideally better in *some* way than the design you selected).

Feasibility
You must include a detailed description of how it is feasible to implement the proposed application. Here are several common factors that could lead to software development failures:

- A lack of familiarity with APIs

- Unable to deliver on performance
- Excessive cost
- Already existing project
- Too many features
- Features that are not useful
- Third party APIs/services that may not be reliable to use

Make sure your Design Doc has:

- **Evidence of novelty:** What purpose does your application serve? If related applications already exist, how is yours different? Are these changes useful or solve an important problem?

- **Documentation of UML diagrams:** Use case diagram(s), sequence diagram(s), and (class diagram(s) or state chart diagram(s)).

- **UI Mock-ups (optional):** you can use Sketch, Figma, InVision (or any other tool of your choice) to make UI Mock-ups to give us an idea of what the application will look like.

- The design doc should be **less than 10 pages**, **inclusive of diagrams, figures, tables, references**. Having more pages will not result in better grades.

Please ensure that:
- User Stories clearly describe a coherent set of functional requirements
- User Stories follow the INVEST principle (and make dependencies explicit)
- Quality Attribute Scenarios include a stimulus and response measure
- Quality Attribute Scenarios clearly describe important non-functional requirements
- Diagrams use notation correctly
- Diagrams are clear and understandable

Presentation

You are going to present your idea in your discussion section *on* Week 4. Each team is scheduled for a **6-minute presentation**.

The goal of the presentation is to communicate your proposed application clearly and convincingly to the teaching team and your classmates. Your presentation should give us enough information to understand what you plan to build, why it is useful, and why it is feasible for your team to complete within the quarter.

Your presentation should cover the following:

Problem and motivation. Explain the user need, pain point, or opportunity that motivates your project. Make it clear who your target users are and why this problem is worth solving.

Project idea and scope. Describe the application you plan to build and the core functionality you expect to complete this quarter. Distinguish your core features from possible stretch goals.

Design overview. Give a high-level overview of your proposed design. This may include the major system components, architecture, data flow, or key technical decisions. You do not need to present every diagram from the design doc.

Mockups and user interaction. Show mockups, wireframes, or sketches of your interface and briefly walk through one usage scenario.

Team plan. Briefly describe how the work will be divided across team members and how your team plans to collaborate.

Checklist

- ❏ Form and sign up your team
- ❏ Finalize an idea for your application
- ❏ Set up a repository on GitHub and add your TA
  Eric: https://github.com/zitongzhoueric,
  Qiwei: https://github.com/Qiwei-Di1234
- ❏ Create mockup designs (e.g., screen snapshots)
- ❏ Write Design Doc
- ❏ Create slides and prepare for the presentation
- ❏ Submit Design Doc & Slides on GradeScope (Only **one per team**, please create a group submission and **add all team members**)